

Arquitetura de Software em Sistemas Distribuídos

Aula 06

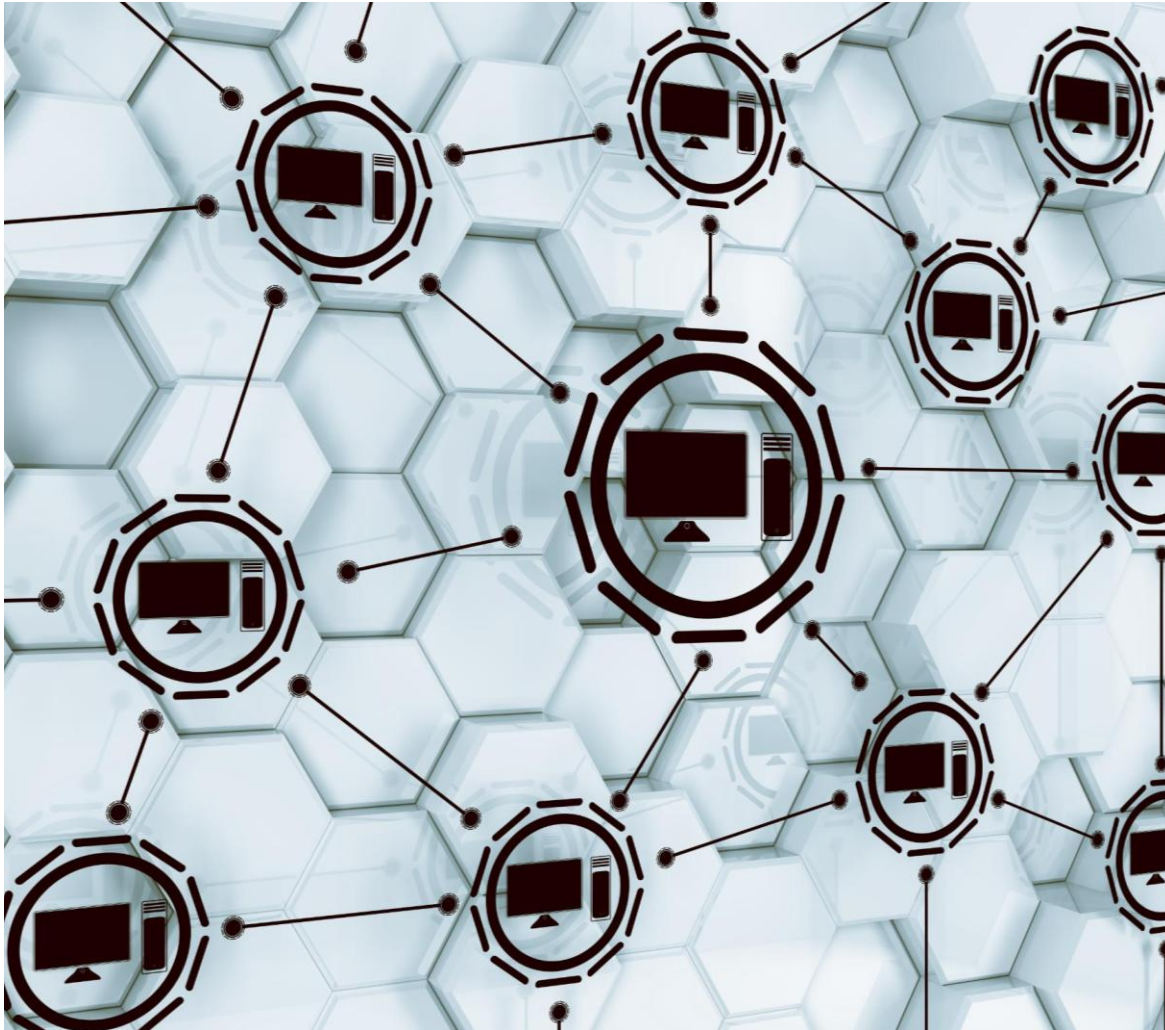
Prof. Paulo Bressan

Agenda da Apresentação

- Fundamentos e Estilos Arquiteturais em Sistemas Distribuídos
 - Definição, importância e requisitos arquiteturais
 - Estilos arquiteturais, componentes e conectores
- Padrões de arquitetura de software em sistemas distribuídos
 1. Cliente-Servidor
 2. Arquitetura em Camadas (N-Tier)
 3. Orientada a Serviços (SOA – Service-Oriented Architecture)
 4. Microserviços
 5. Arquitetura Baseada em Eventos (Event-Driven)
 6. Arquitetura Peer-to-Peer (P2P)
 7. Arquitetura Baseada em DHT (Distributed Hash Table)
- Comparação dos estilos arquiteturais
- Conclusão

Fundamentos e Estilos Arquiteturais em Sistemas Distribuídos

Definição e importância da arquitetura de software distribuída



Organização de Componentes Distribuídos

Arquitetura define como componentes se organizam em diferentes máquinas para operar conjuntamente.

Desempenho e Escalabilidade

Arquiteturas distribuídas garantem alto desempenho e escalabilidade em sistemas distribuídos.

Robustez em Ambientes Heterogêneos

Garantem robustez e confiabilidade mesmo em ambientes geograficamente dispersos e variados.

Meta importante

Separar aplicações das plataformas subjacentes provendo uma camada de middleware

Requisitos arquiteturais



Escalabilidade

Arquiteturas devem suportar crescimento de carga para atender a demanda crescente sem perda de desempenho.

Disponibilidade

Garantir funcionamento contínuo do sistema, mesmo em situações adversas ou alta demanda.

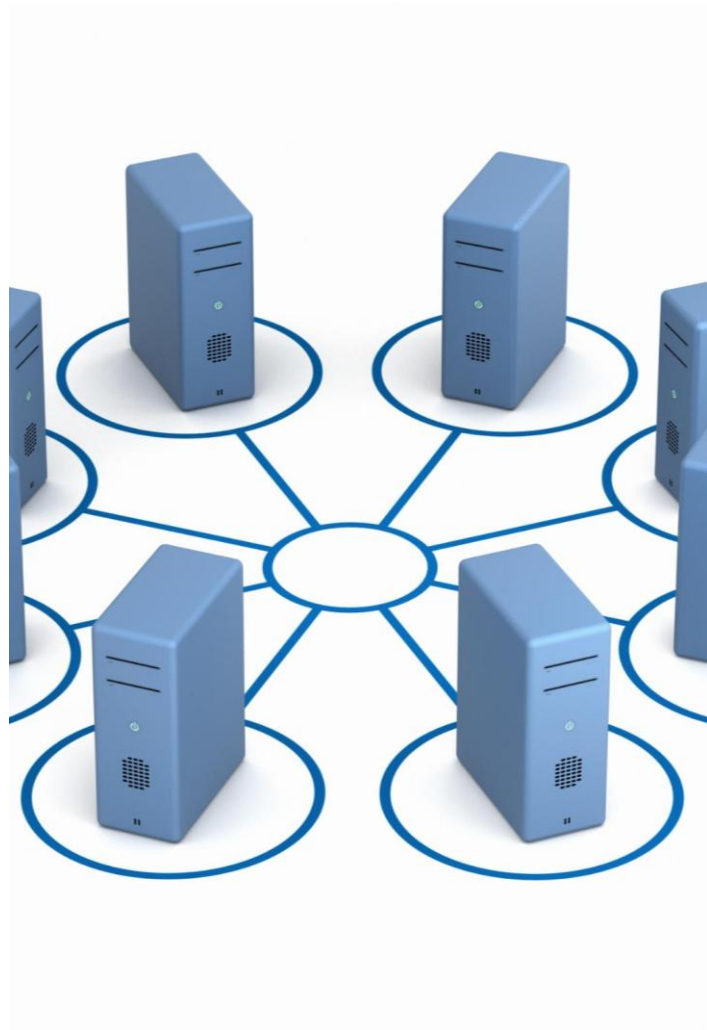
Tolerância a Falhas

Resistência a falhas permite que o sistema mantenha operação mesmo com erros ou falhas parciais.

Consistência e Segurança

Manutenção da integridade dos dados e aplicação de políticas robustas de segurança são essenciais.

Estilos arquiteturais



Os estilos arquiteturais em sistemas distribuídos representam padrões de organização que orientam a forma como um sistema é estruturado e como seus componentes – sejam eles processos, serviços ou nós de rede – se relacionam.

Eles determinam:

como ocorre a comunicação (síncrona, assíncrona, orientada a mensagens, via chamadas de procedimento remoto, etc.),

a **distribuição das responsabilidades** entre os elementos do sistema (como no modelo cliente-servidor ou em microsserviços),

e a **forma de coordenação** para que os diferentes módulos atuem de maneira integrada.

Além disso, cada estilo busca atender a um conjunto de **requisitos de qualidade** do sistema, como **desempenho, escalabilidade, disponibilidade e segurança**.

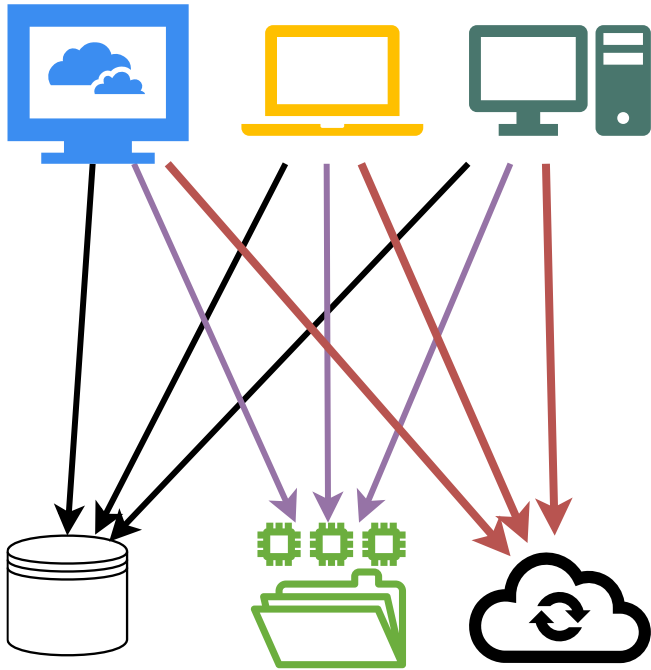
Estilos arquiteturais: Componentes e Conectores



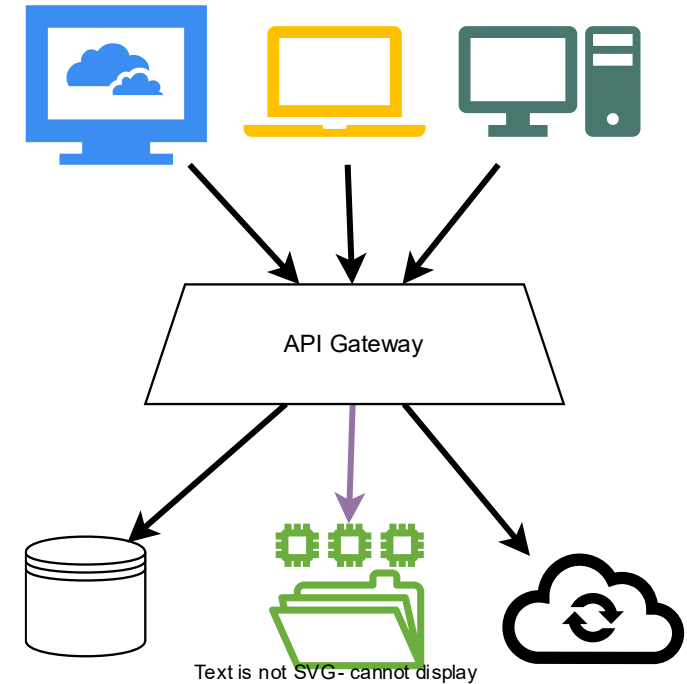
- Os componentes são conectados por meio de conectores, que podem assumir múltiplas formas para esconder as complexas iterações entres os componentes, por exemplo, por meio de fluxos de mensagens ou invocações remotas de procedimentos.
- Quando houver forte dependência, diremos que os componentes estão **fortemente acoplados** (*tightly coupled*).
- Caso contrário, diremos que estão **fracamente acoplados** (*loosely coupled*).

Estilos arquiteturais: Componentes e Conectores

SEM API GATEWAY



COM API GATEWAY

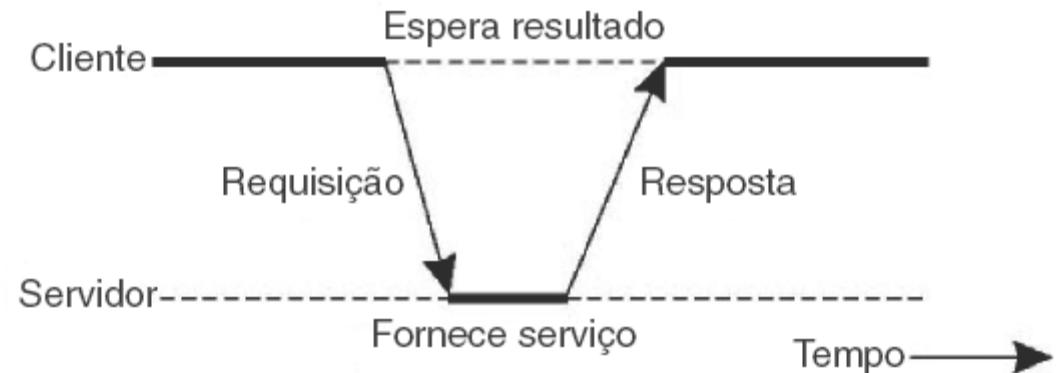


Padrões de arquitetura de software em sistemas distribuídos



1. Cliente-Servidor

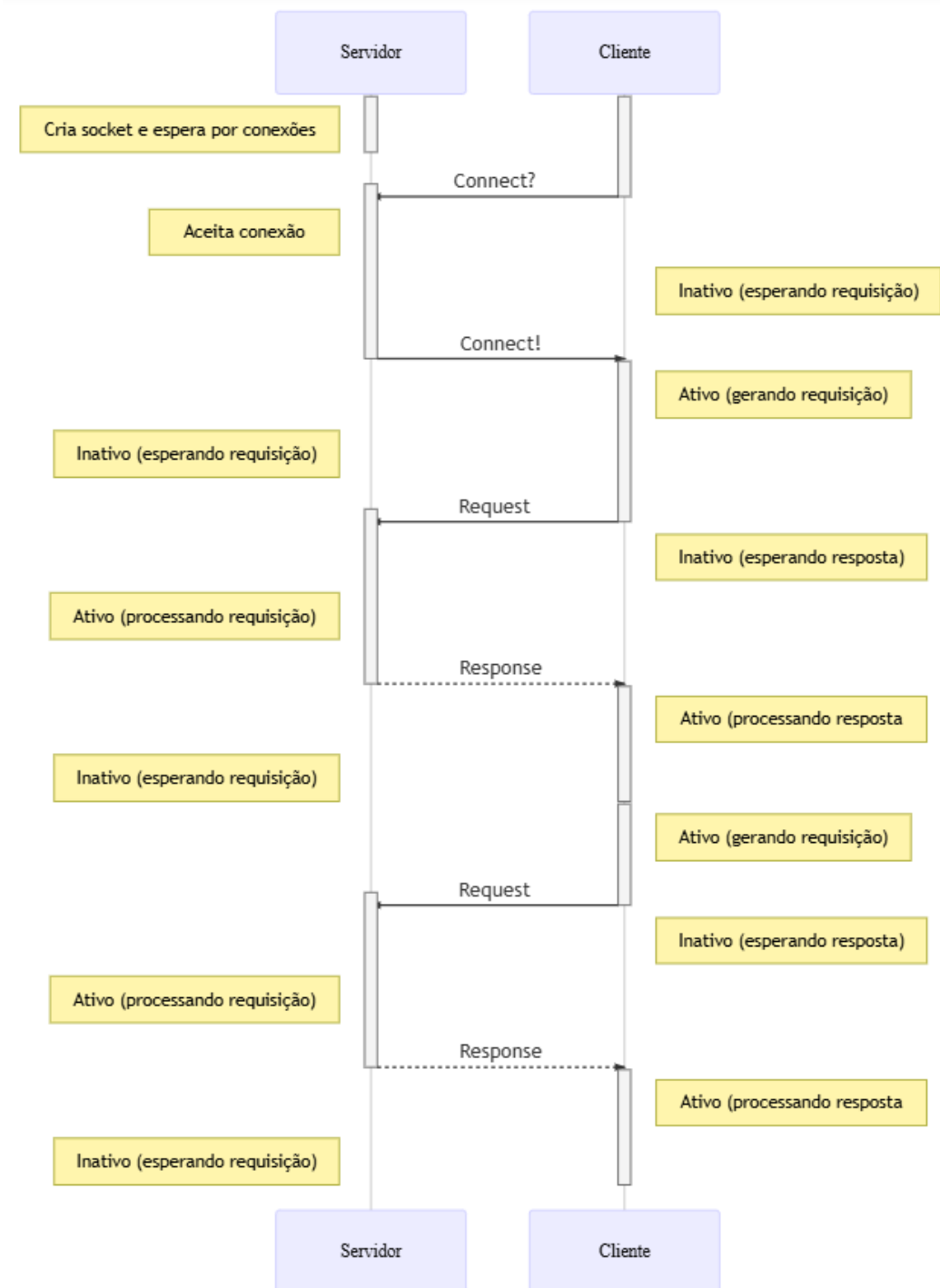
- **Definição:** separa o sistema em **clientes** (fazem requisições) e **servidores** (processam e respondem).
- **Exemplo:** navegador web (cliente) ↔ servidor web (Apache, Nginx).
- **Pontos fortes:** simplicidade, centralização de dados.
- **Limitações:** gargalo e ponto único de falha no servidor.



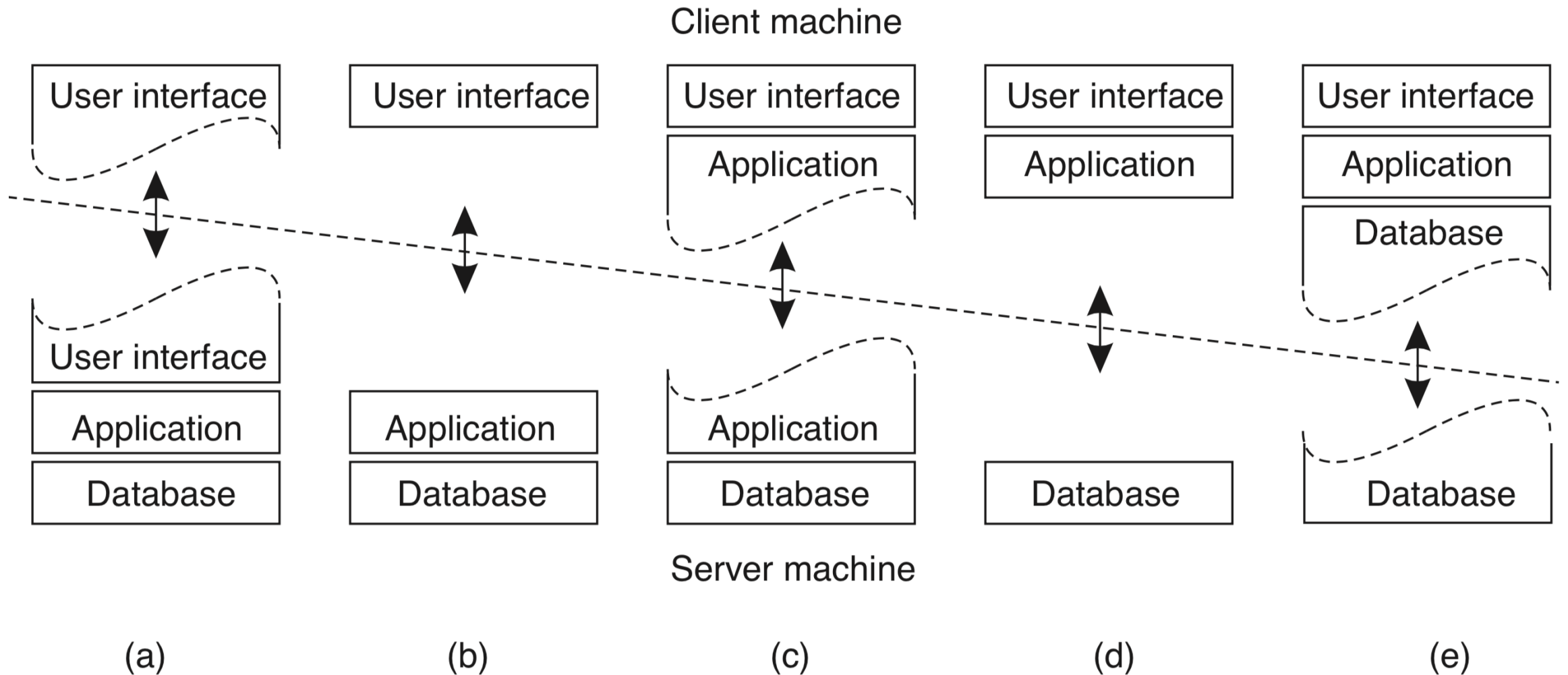
1. Cliente-Servidor

Embora seja possível usar sockets de forma assíncrona, a API mais comum é síncrona.

Quando um processo espera receber uma mensagem de outro, ele fica bloqueado esperando algum dado estar disponível para leitura no referido socket.

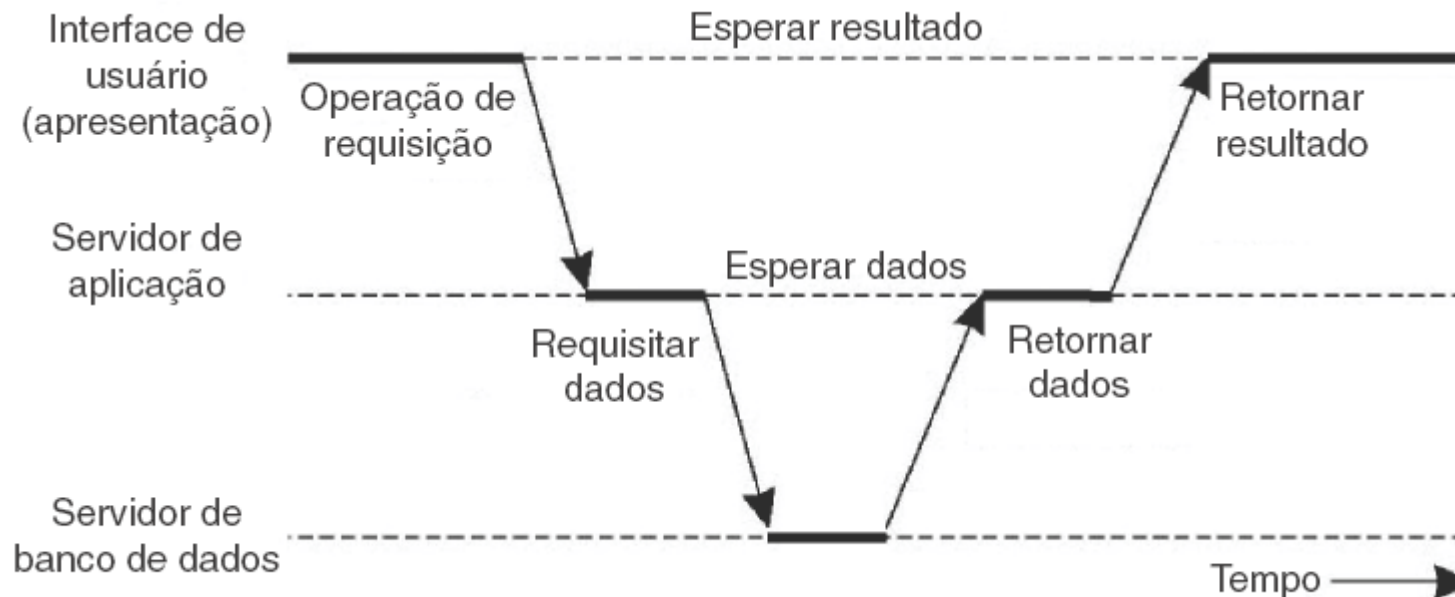


1. Cliente-Servidor

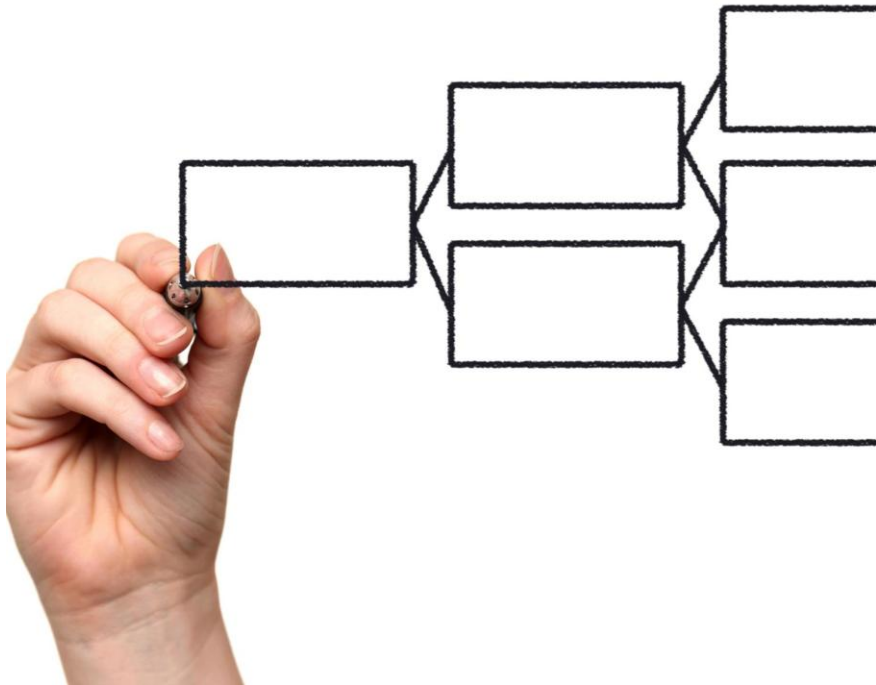


1. Cliente-Servidor

- Como estabelecer uma distinção clara entre um cliente e um servidor?
- Por exemplo, um servidor para um banco de dados distribuído pode agir como um cliente repassando requisições para diferentes servidores de arquivos.



2. Arquitetura em Camadas (N-Tier)



Definição

organiza o sistema em camadas hierárquicas (apresentação, lógica de negócio, persistência de dados).

Exemplo

aplicações empresariais com frontend → middleware → banco de dados.

Pontos fortes

modularidade, facilidade de manutenção.

Limitações

latência devido ao número de camadas, menor flexibilidade em grandes escalas.

2. Arquitetura em Camadas (N-Tier)

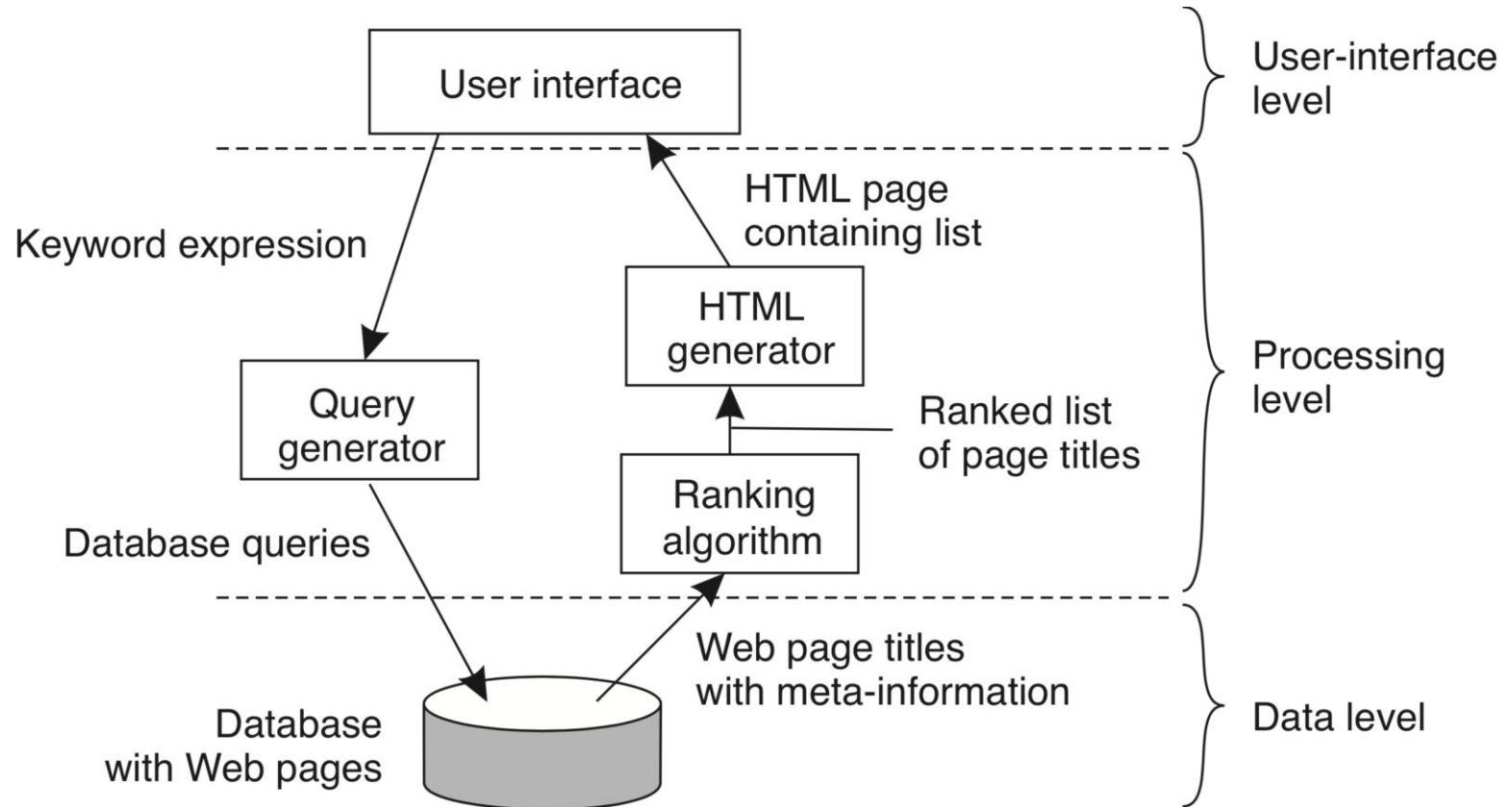
1. Nível de interface do usuário:
gerenciamento de exibição

2. Nível de processamento: contém
as aplicações

3. Nível de dados: os dados aonde
está sendo executada uma ação

Exemplo:

- mecanismos de busca pela internet
- suporte a decisão de corretora de valores





3. SOA – Service-Oriented Architecture

Definição

conjunto de **serviços independentes**, que se comunicam por protocolos padronizados (SOAP/REST).

Exemplo

sistemas corporativos que integram ERP, CRM e sistemas legados via serviços.

Pontos fortes

reuso, integração de sistemas heterogêneos.

Limitações

complexidade de governança, sobrecarga de comunicação.

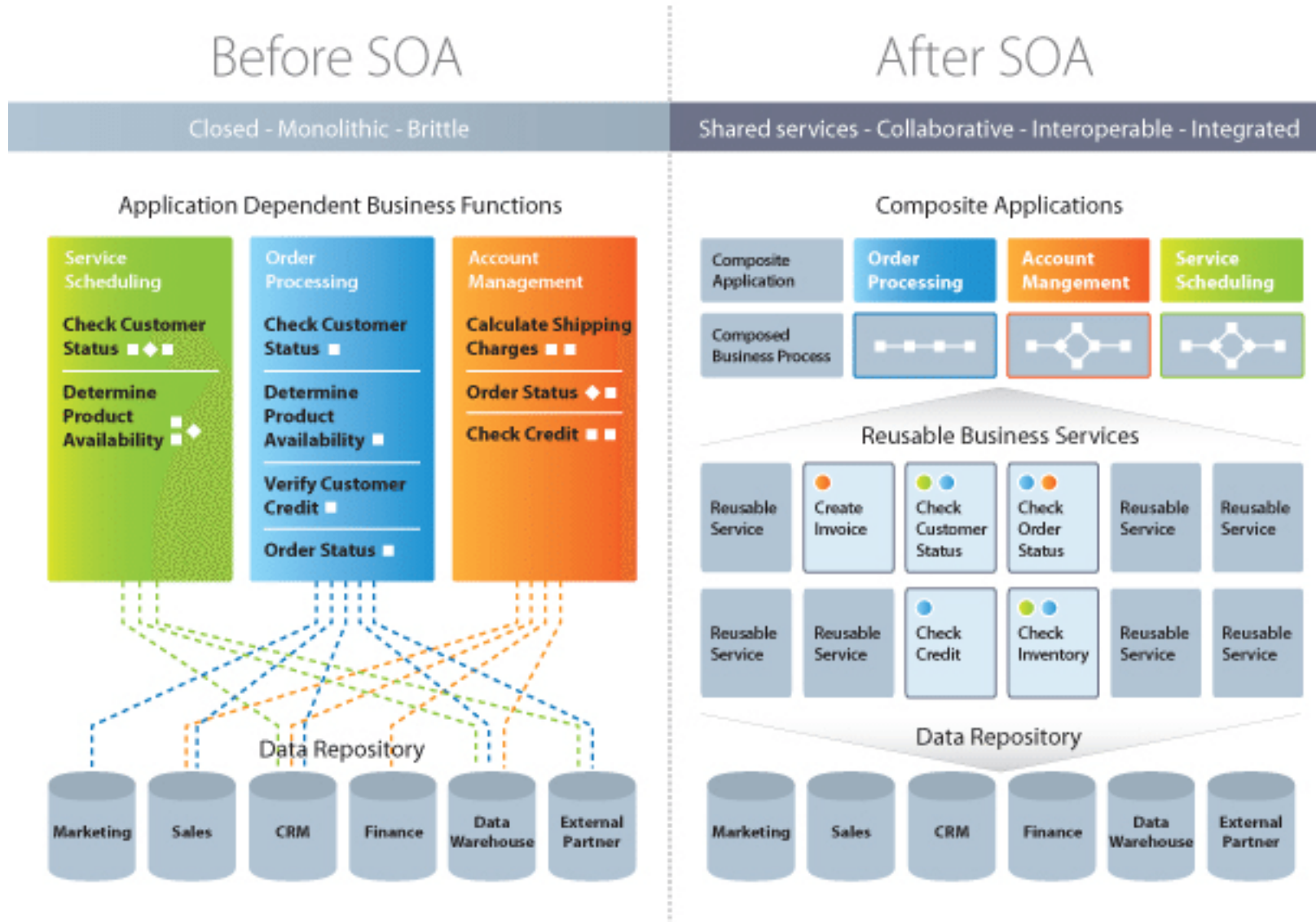
3. SOA – Service-Oriented Architecture

Conceitos-chave do SOA

- **Serviço:** uma unidade funcional autônoma, que realiza uma tarefa específica (ex.: “processar pagamento”, “consultar estoque”, “calcular frete”).
- **Interface bem definida:** serviços se comunicam por meio de interfaces padronizadas, geralmente utilizando protocolos independentes de plataforma (SOAP, REST, XML, JSON).
- **Acoplamento fraco:** os serviços são independentes entre si, facilitando substituição, atualização e evolução sem comprometer o restante do sistema.
- **Reuso:** um mesmo serviço pode ser utilizado por diferentes aplicações.



3. SOA – Service-Oriented Architecture



3. SOA – Service-Oriented Architecture

Componentes típicos

Provedor de Serviço (Service Provider)

Implementa e disponibiliza o serviço.

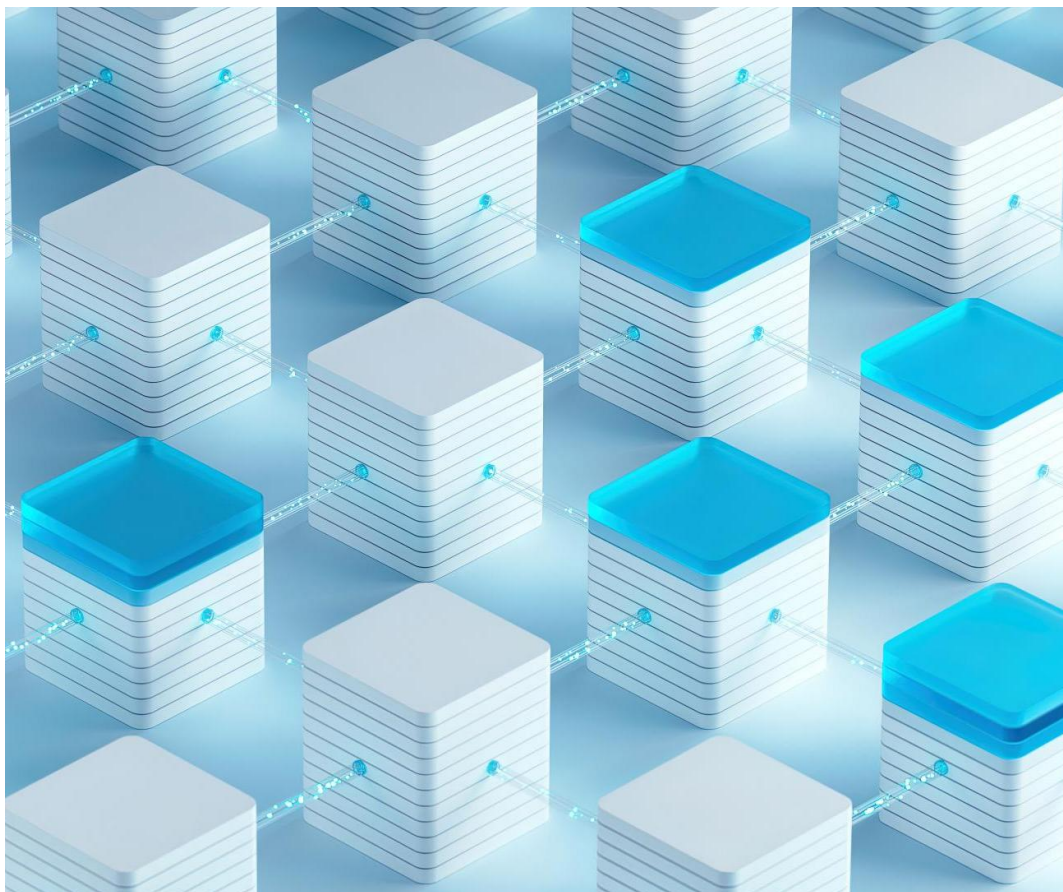
Consumidor de Serviço (Service Consumer)

Aplica ou invoca o serviço, geralmente via rede.

Registro de Serviços (Service Registry)

Diretório central que lista serviços disponíveis, permitindo descoberta dinâmica.





4. Microserviços

Definição

evolução do SOA, cada **serviço é pequeno, autônomo e independente**, geralmente comunicando-se por APIs REST ou mensageria.

Exemplo

Netflix, Amazon, Spotify.

Pontos fortes

escalabilidade independente, deploy contínuo.

Limitações

complexidade na orquestração, monitoramento e segurança.

4. Microserviços

Conceitos-chave

Serviços independentes

Cada microserviço é responsável por uma funcionalidade única, como “gerenciar usuários”, “processar pagamentos” ou “gerar relatórios”.

Desacoplamento

Microserviços se comunicam por **APIs** (REST, gRPC) ou mensagens assíncronas (filas, eventos), minimizando dependências diretas.

Autonomia e Deploy Independente

Cada serviço pode ser atualizado ou escalado **sem impactar os demais serviços** do sistema.

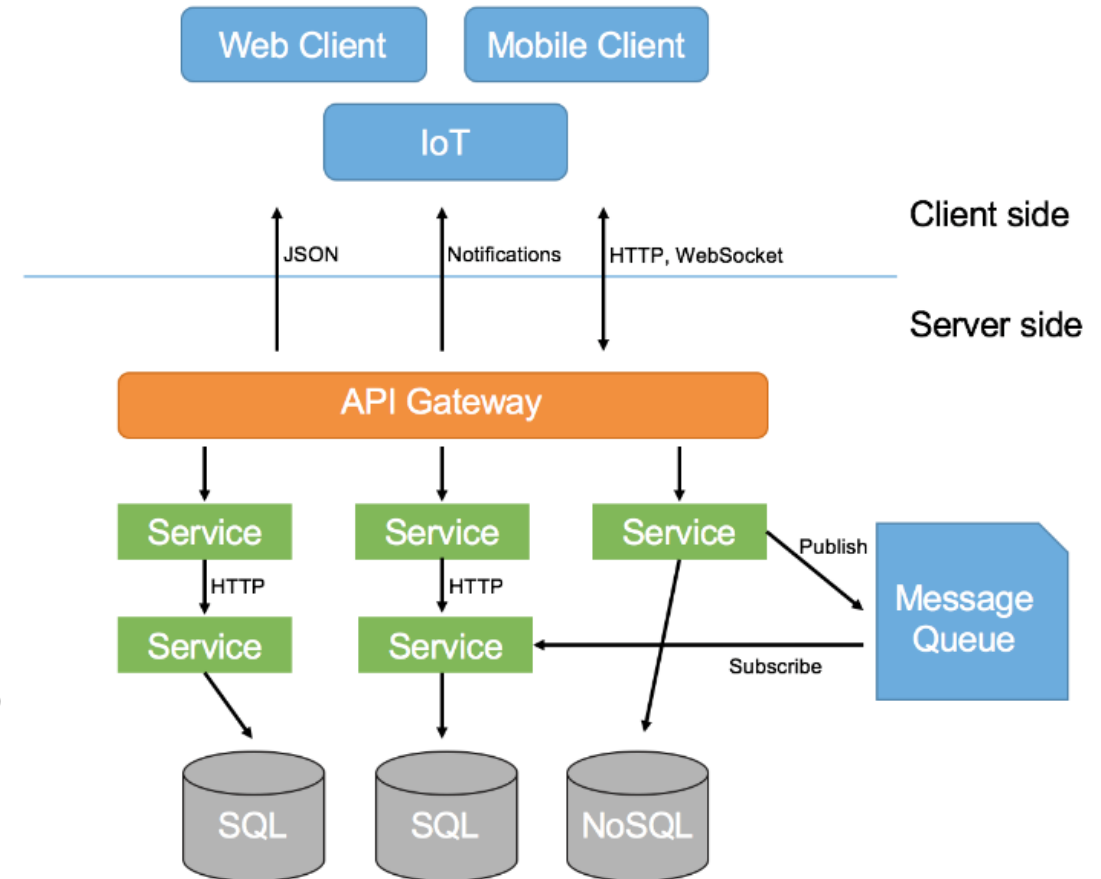
Banco de dados independente

Cada microserviço pode ter seu próprio repositório de dados, evitando acoplamento no armazenamento.

Comunicação

Síncrona: chamadas HTTP/REST ou gRPC.

Assíncrona: filas, tópicos ou eventos (Kafka, RabbitMQ).



4. Microserviços

Benefícios

- Escalabilidade granular: apenas os serviços que necessitam podem ser replicados.
- Flexibilidade tecnológica: cada serviço pode usar linguagens ou bancos de dados diferentes.
- Resiliência: falha em um serviço não derruba o sistema inteiro.
- Agilidade: facilita integração contínua, deploy contínuo e times independentes.

Desafios e Limitações

- Complexidade operacional: monitoramento, logging e deploy de muitos serviços.
- Gerenciamento de comunicação: problemas de latência, tolerância a falhas e consistência eventual.
- Orquestração: necessidade de um gateway ou barramento de serviços para coordenar microserviços.
- Segurança: múltiplas APIs expostas exigem controle rigoroso de autenticação e autorização.

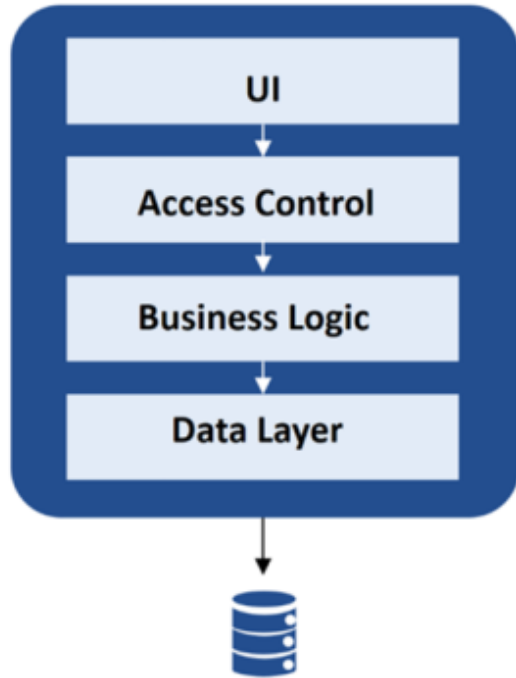
Exemplos práticos

- Netflix: microserviços independentes para catalogar filmes, recomendar conteúdo e processar streaming.
- Amazon: serviços separados para catálogo de produtos, carrinho, pagamentos e logística.
- Spotify: microserviços para playlists, reprodução, recomendações e análise de dados.

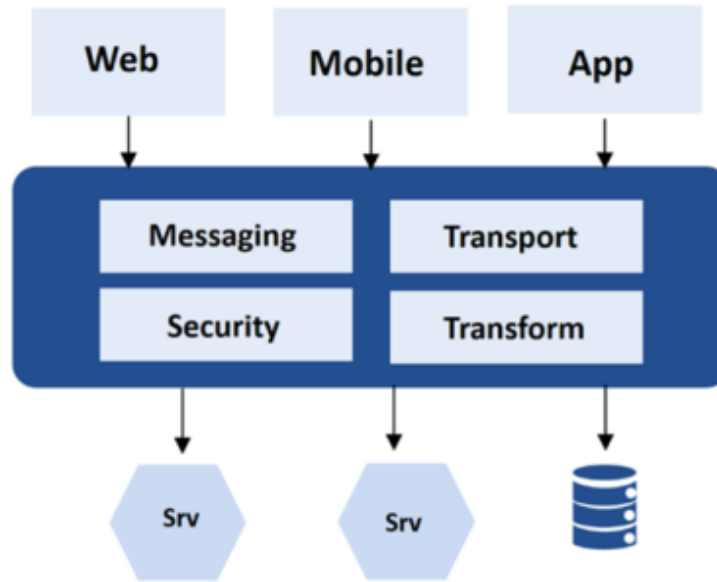
Diferença entre SOA e Microserviços

Aspecto	SOA	Microserviços
Tamanho do serviço	Grande, integrado	Pequeno, focado
Banco de dados	Compartilhado	Independente por serviço
Comunicação	Principalmente síncrona	Síncrona ou assíncrona
Escalabilidade	Geral	Granular por serviço
Complexidade	Governança centralizada	Maior complexidade operacional

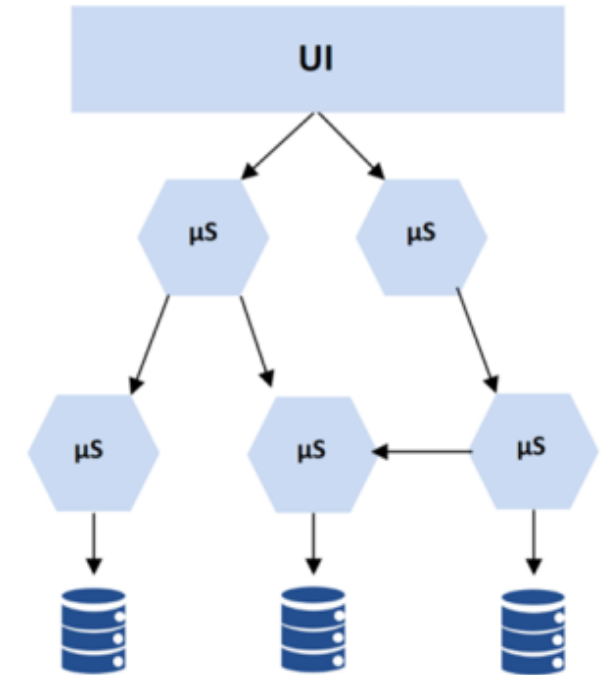
Evolution of Software Architectures



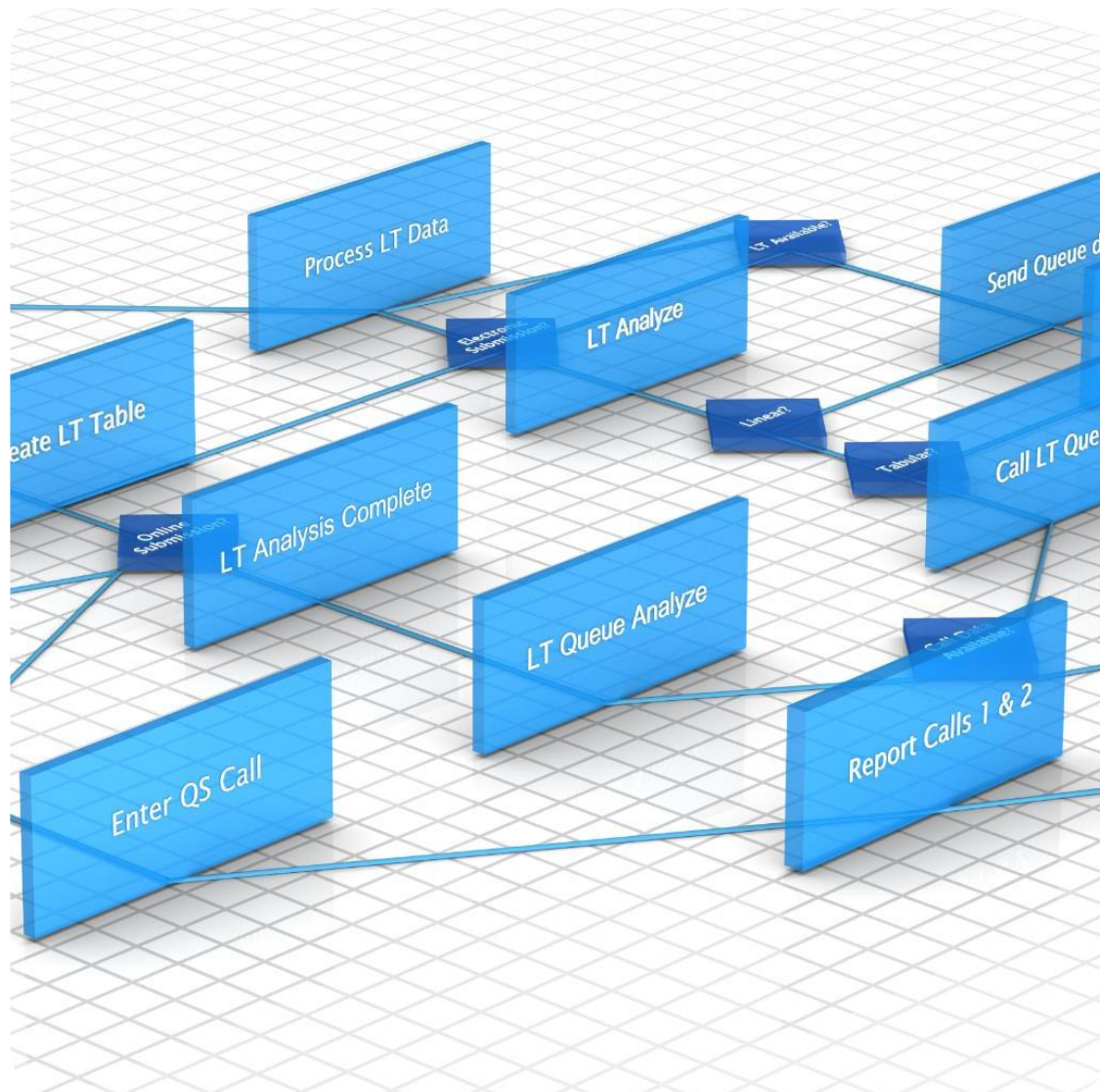
Monolithic



Service-oriented



Microservices



5. Arquitetura Baseada em Eventos (Event-Driven)

Definição

os componentes se comunicam de forma **assíncrona** por meio de **eventos** publicados em filas ou barramentos.

Exemplo

sistemas de pagamento, IoT, mensageria (Kafka, RabbitMQ).

Pontos fortes

desacoplamento, alta escalabilidade.

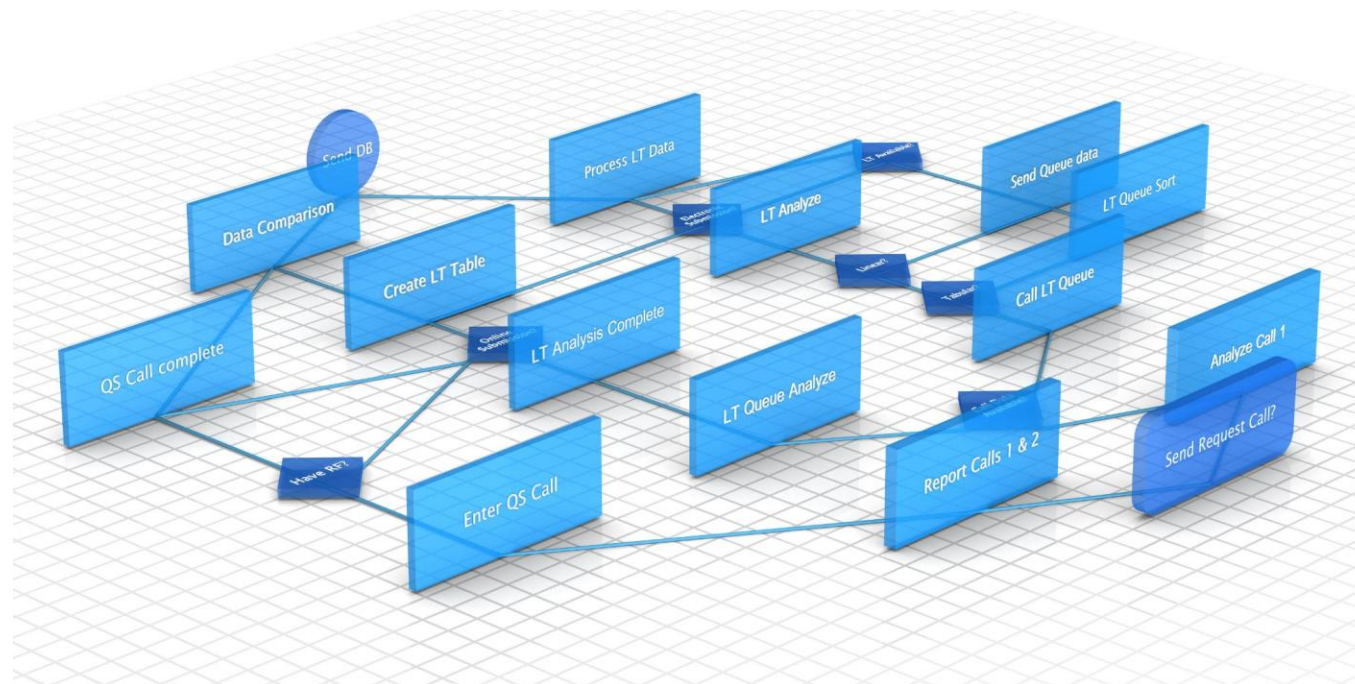
Limitações

maior dificuldade de rastreamento e consistência.

Arquitetura baseada em eventos

Em vez de chamadas diretas entre serviços, o sistema funciona com a lógica de “**produção** → **detecção** → **consumo**” de eventos.

Um **evento** é qualquer ocorrência significativa que represente uma mudança de estado no sistema, como “*pedido criado*”, “*pagamento aprovado*”, “*sensor detectou movimento*”.



Arquitetura baseada em eventos

Como funciona:

1. Produção de Eventos:

Um serviço detecta uma mudança de estado (ex: um item adicionado ao carrinho) e gera um "evento" para notificar outros componentes.

2. Publicação de Eventos:

O evento é publicado em um canal de comunicação, como um barramento de eventos ou stream de dados.

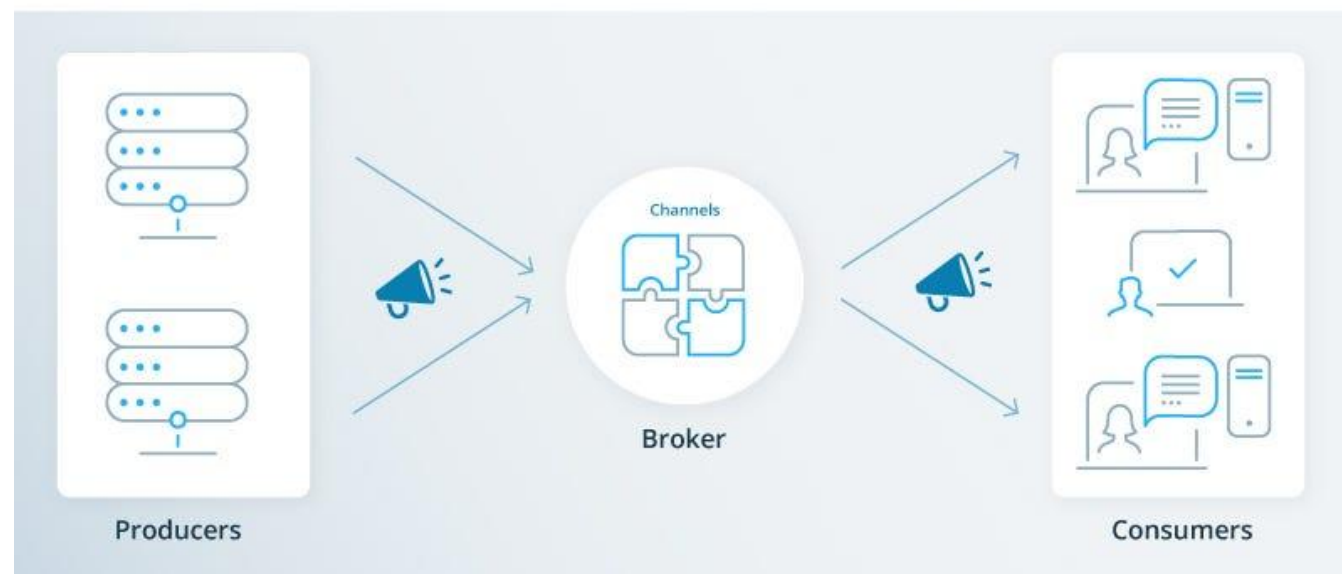
3. Encaminhamento e Roteamento:

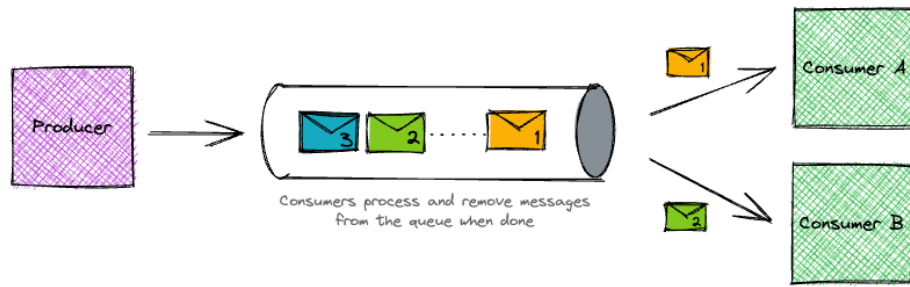
O canal de comunicação encaminha o evento para os consumidores que se inscreveram para receber esse tipo específico de evento.

4. Consumo e Reação:

Os consumidores recebem o evento e executam suas lógicas de negócio, que podem gerar outras ações ou novos eventos, e assim por diante.

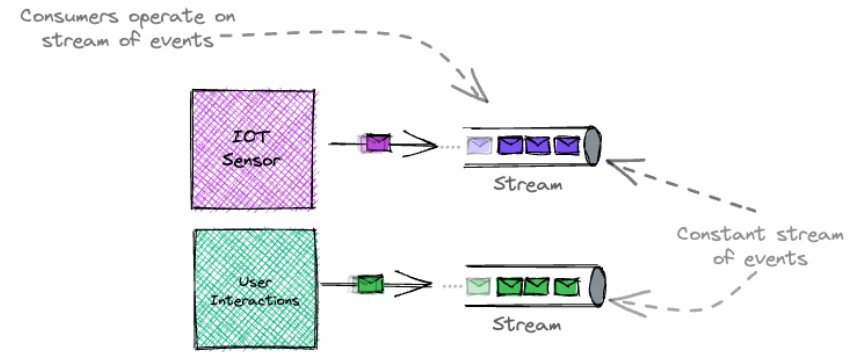
Event-Driven Architecture





Queues

Consumers pull messages, process and remove



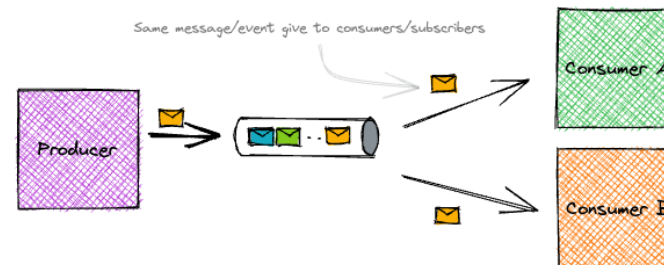
Streams

Unbounded series of events
Common examples include user click streams / IOT / transactions

Queues vs Streams vs Pub/Sub

Bite sized visual to help understand the differences

@boyney123



Pub/Sub

Publish messages/events to many subscribers
Each subscriber gets copy of event to process

5. Arquitetura Baseada em Eventos (Event-Driven)

Benefícios

Desacoplamento: produtores não precisam saber quem consome os eventos.

Escalabilidade: permite lidar com grandes volumes de dados e picos de requisições.

Reatividade: sistemas respondem em tempo real a mudanças e notificações.

Flexibilidade: novos consumidores podem ser adicionados sem modificar os produtores.

Desafios e Limitações

Complexidade de rastreamento: difícil garantir ordem e rastreabilidade de eventos em sistemas distribuídos.

Consistência eventual: o estado do sistema pode levar tempo até se alinhar.

Monitoramento e debugging: mais complexos do que em arquiteturas síncronas.

Garantia de entrega: é preciso lidar com duplicações, perda de eventos ou reprocessamentos.

Exemplos práticos

E-commerce:

Evento: “*pedido criado*” → aciona serviços de pagamento, estoque e entrega.

IoT:

Sensores enviam eventos continuamente (temperatura, movimento, umidade).

Finanças:

Transações bancárias geram eventos para sistemas antifraude e auditoria em tempo real.

Streaming (Netflix, Spotify):

Eventos de “play”, “pause” ou “skip” geram dados para análise e recomendação.

6. Arquitetura Peer-to-Peer (P2P)

Definição

não há servidores centrais fixos; todos os nós podem atuar como clientes e servidores.

Exemplo

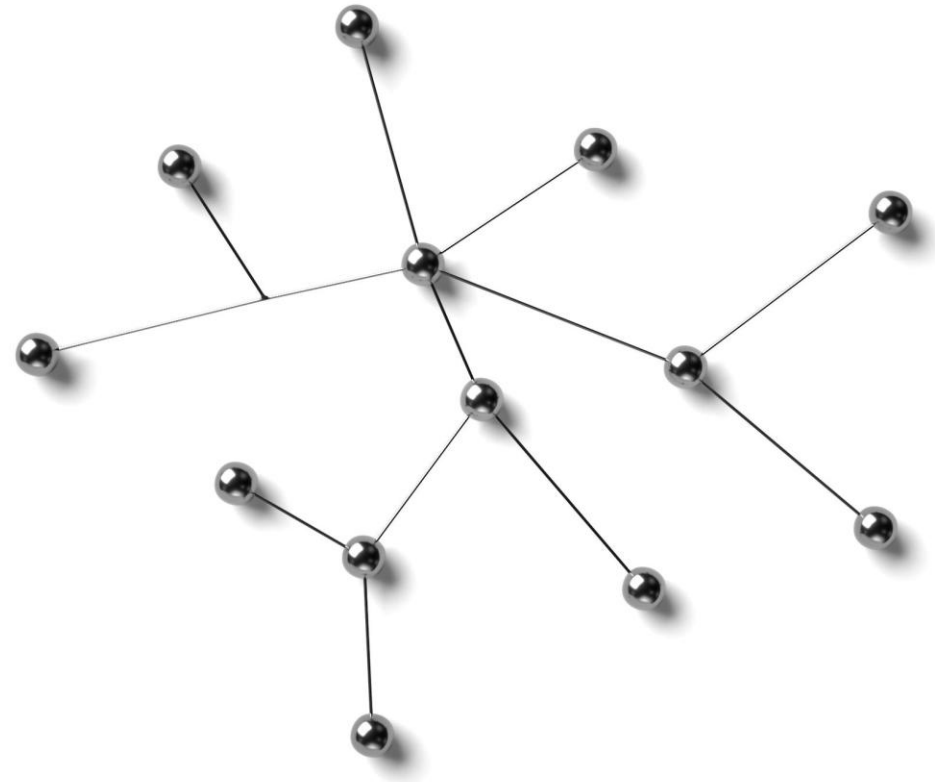
BitTorrent, blockchain, redes de compartilhamento de arquivos.

Pontos fortes

tolerância a falhas, descentralização.

Limitações

dificuldade de coordenação, segurança e confiabilidade.



6. Arquitetura Peer-to-Peer (P2P)

Um sistema peer-to-peer implementa uma rede abstrata sobreposta, em cima da topologia da rede.

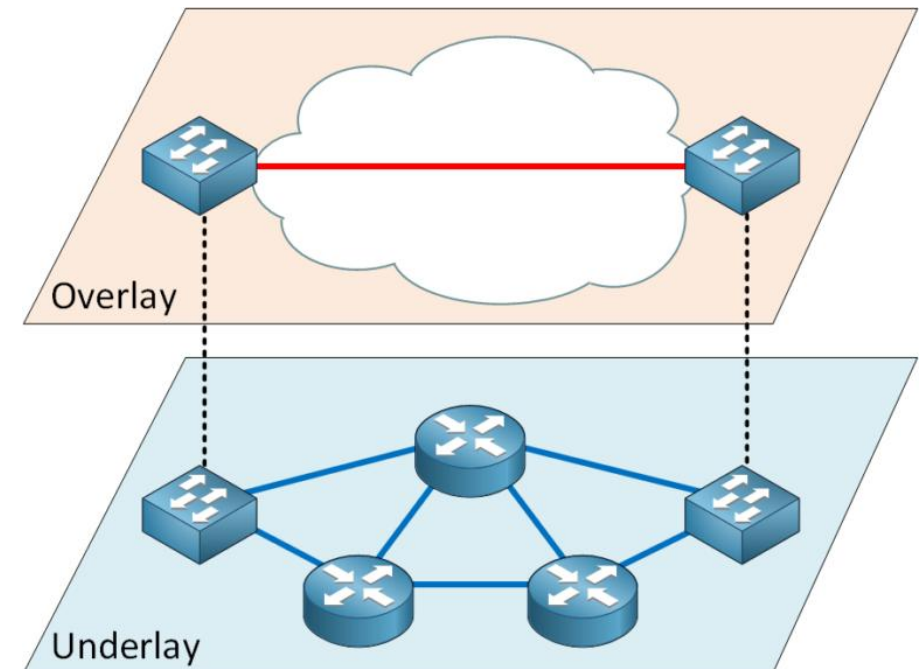
Essa sobreposição é utilizada para descobrir e indexar os pares da rede, tornando o sistema P2P funcional, independente da topologia da rede física.

O conteúdo é trocado diretamente sobre o protocolo IP.

Essa arquitetura depende da alta performance da internet.

Como os recursos necessários são distribuídos (carga computacional, tráfego de rede, espaço de armazenamento, etc.) entre os nós, é possível conseguir um melhor desempenho de forma econômica.

Diferentemente da rede cliente-servidor, onde a performance depende do desempenho do servidor.



6. Arquitetura Peer-to-Peer (P2P)

Benefícios

Descentralização: sem ponto único de falha.

Escalabilidade: mais usuários = mais capacidade de rede.

Resiliência: a rede continua funcionando mesmo que vários peers saiam.

Distribuição de carga: processamento e dados distribuídos entre nós.

Desafios e Limitações

Segurança: difícil controlar acesso e verificar a autenticidade dos peers.

Desempenho variável: depende da conectividade e disponibilidade dos peers.

Gerenciamento de dados: manter consistência e disponibilidade pode ser complexo.

Roteamento e descoberta de recursos: exige protocolos inteligentes para localizar peers e conteúdos.

Exemplos práticos

Compartilhamento de arquivos: BitTorrent, eMule.

Criptomoedas e Blockchain: Bitcoin, Ethereum (cada nó valida e replica transações).

VoIP: Skype (nas versões iniciais usava P2P para chamadas e transmissão de dados).

Computação distribuída voluntária: SETI@home, Folding@home (uso da CPU de múltiplos peers).



eMule v0.46b

Disconnect Kad Servers Transfers Search Shared Files Messages IRC Statistics Options Tools Help

Downloads (10)

File Name	Comple...	Speed	Progress	Size	Sources	Priority	Remaining
Blender-2.37a-Windows.[content.emule-project....	1.71 MB			4.65 MB	13	Normal	6:31 h (2.94 MB)
Hi-res textures for use with modern Quake engines (...)	510.00 KB			83.67 MB	14	Normal	? (83.17 MB)
KNOPPIX_V3.9-2005-05-27-EN.[content.emule-p...	531.81 MB	207.25 KB/s		695.89 MB	223/231 (10)	Normal	13:30 mins (164.0...
Maestro-Win.[content.emule-project.net].exe	9.63 MB	9.25 KB/s		38.51 MB	18 (2)	Normal	53:18 mins (28.88 ...
opensource_cd23.[content.emule-project.net].iso	7.84 MB	4.98 KB/s		554.55 MB	22 (1)	Normal	1 D 7 h (546.72 MB)
selfhtml81.zip	8.08 MB			8.08 MB		Normal	
the_scene_xvid_episode_7.avi	2.43 MB			61.06 MB	30	Normal	3 D 17 h (58.64 MB)
the_scene_xvid_episode_8.avi	12.68 MB	7.29 KB/s		118.60 MB	29 (2)	Normal	4:08 h (105.92 MB)
the_scene_xvid_episode_9.avi	3.59 MB	4.96 KB/s		39.45 MB	33 (2)	Normal	2:03 h (35.86 MB)
wikipedia_cd_version.1.0.iso	14.67 MB			668.63 MB	34/35	Normal	8 D 3 h (653.95 MB)

Uploading (12)

User Name	File	Upload...	Obtained Parts	Status	
ZOTON	KNOPPIX_V3.9-2005-05-27-EN.[content.emule-project.net].iso	1:23 ...		Transferring	
pepuk	KNOPPIX_V3.9-2005-05-27-EN.[content.emule-project.net].iso	3:08 ...		Transferring	
PePies	KNOPPIX_V3.9-2005-05-27-EN.[content.emule-project.net].iso	:21 mins		Transferring	
ippsatsi	KNOPPIX_V3.9-2005-05-27-EN.[content.emule-project.net].iso	0:55 ...		Transferring	
http://emule-project.net	KNOPPIX_V3.9-2005-05-27-EN.[content.emule-project.net].iso	7:12 ...		Transferring	
http://emule-project.net	KNOPPIX_V3.9-2005-05-27-EN.[content.emule-project.net].iso	0:07 ...		Transferring	
http://emule-project.net	KNOPPIX_V3.9-2005-05-27-EN.[content.emule-project.net].iso	:10 mins		Transferring	
http://emule-project.net	KNOPPIX_V3.9-2005-05-27-EN.[content.emule-project.net].iso	:16 mins		Transferring	
http://emule-project.net	KNOPPIX_V3.9-2005-05-27-EN.[content.emule-project.net].iso	0 B/s	359.46 KB	2:19 mins	Transferring
eMule0.42e27.Sivka....	KNOPPIX_V3.9-2005-05-27-EN.[content.emule-project.net].iso	0 B/s	35.29 KB	47 secs	Transferring
bob22	KNOPPIX_V3.9-2005-05-27-EN.[content.emule-project.net].iso	1.96 KB/s	1.23 MB	6:52 mins	Transferring
ANTONIO10NIN	KNOPPIX_V3.9-2005-05-27-EN.[content.emule-project.net].iso	13.41 KB/s	4.15 MB	8:07 mins	Transferring

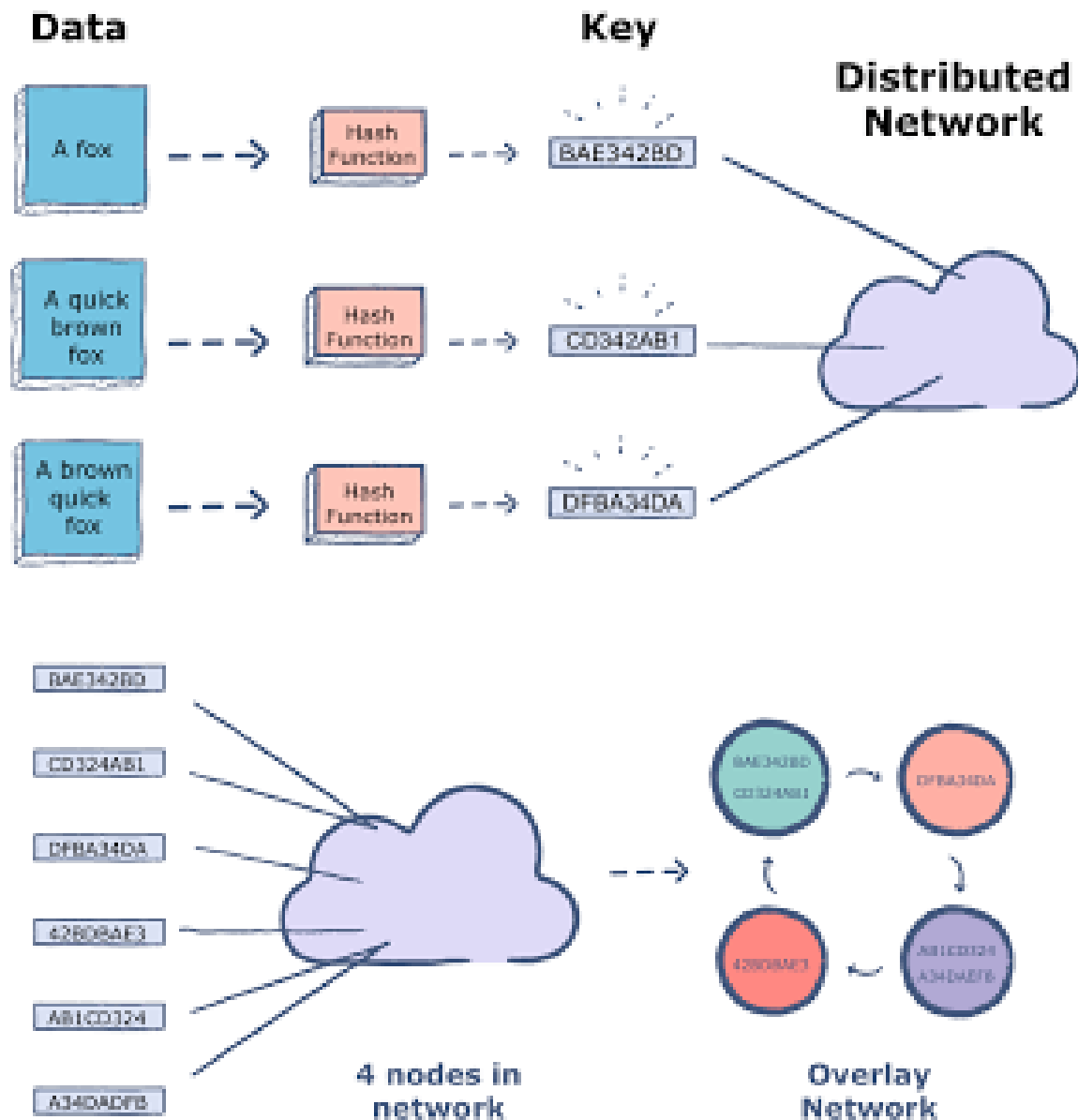
Downloads menu:

- Priority (Download)
- Pause
- Stop
- Resume
- Cancel
- Open File
- Preview
- Details...
- Comments...
- Clear Completed
- eD2K Links...
- Paste eD2K Links
- Web Services
- Assign To Category

Clients on queue: 4 (1 banned)

Finished downloading selfhtml81.zip :-)

Users:5.3 M(2.2 M)|Files:573.5 M(363.2 M) Up: 49.3 | Down: 233.5 eD2K:Connected|Kad:Connected



7. Arquitetura Baseada em DHT (Distributed Hash Table)

Definição

organiza os nós em uma **estrutura de tabela de hash distribuída**, facilitando buscas e armazenamento distribuído.

Exemplo

sistemas de armazenamento distribuído (Kademlia, Chord, DynamoDB).

Pontos fortes

escalabilidade massiva, busca eficiente.

Limitações

complexidade de implementação e manutenção.

Protocolo Chord

1. Identificação de Nós e Chaves

- ✓ Cada nó e chave recebem um ID por hash (ex.: SHA-1)
- ✓ IDs formam um anel lógico (0 até $2^m - 1$)
- ✓ Chave é armazenada no primeiro nó com ID $\geq$ hash da chave

2. Armazenamento de Dados

- ✓ Cada chave pertence ao seu sucessor no anel
- ✓ Sucessor = primeiro nó no anel com ID $\geq$ hash da chave
- ✓ Distribuição uniforme das chaves

3. Busca de Dados

- ✓ Para buscar uma chave: encontrar o sucessor do hash
- ✓ Busca direta seria $O(N)$
- ✓ Chord usa a Finger Table para otimizar ($O(\log N)$)

4. Finger Table

- ✓ Cada nó mantém até m entradas (m = bits do hash)
- ✓ Cada entrada aponta para um nó a 2^i posições
- ✓ Permite saltar nós e reduzir buscas
- ✓ Exemplo: do nó 10 para buscar 57 $\rightarrow$ saltos rápidos até 57

5. Entrada e Saída de Nós

- ✓ Novo nó calcula ID e assume parte das chaves
- ✓ Nó que sai: sucessor assume suas chaves
- ✓ Estabilização periódica atualiza ponteiros e finger tables

Protocolo Chord

6. Vantagens e Limitações

Vantagens:

- ✓ Escalável (busca $O(\log N)$)
- ✓ Descentralizado e balanceado
- ✓ Suporta dinamismo de nós

Limitações:

- ✓ Manutenção custosa com muito churn
- ✓ Latência em redes geograficamente distribuídas
- ✓ Vulnerável a ataques de nós maliciosos

7. Exemplo Prático

- ✓ Nós no anel: 10, 30, 50, 70, 90, 110, 130, 150
- ✓ Chave $\text{hash}(X) = 32 \rightarrow$ armazenada no nó 50
- ✓ Cliente no nó 10 procura 32 $\rightarrow$ usa finger table: 10 $\rightarrow$ 30 $\rightarrow$ 50

Comparação dos Estilos Arquiteturais em Sistemas Distribuídos

Estilo Arquitetural	Definição	Exemplos	Vantagens	Limitações
Cliente-Servidor	Cliente requisita serviços; servidor responde	Navegador ↔ Servidor Web, cliente SQL ↔ servidor DB	Simplicidade, centralização de dados e segurança	Ponto único de falha, gargalo do servidor
Em Camadas (N-Tier)	Organização em camadas (apresentação, lógica, dados)	Aplicações corporativas (ERP, sistemas web com backend e DB)	Modularidade, manutenção facilitada, separação de responsabilidades	Latência entre camadas, menor flexibilidade
SOA (Service-Oriented Architecture)	Conjunto de serviços independentes integrados via protocolos	Integração entre ERP, CRM, sistemas legados	Reuso de serviços, integração heterogênea	Complexidade de governança, sobrecarga de comunicação
Microserviços	Serviços pequenos, independentes e autônomos	Netflix, Amazon, Spotify	Escalabilidade independente, deploy contínuo, agilidade	Complexidade em orquestração, monitoramento e segurança
Baseada em Eventos (Event-Driven)	Comunicação assíncrona via eventos, filas, publish/subscribe	Kafka, RabbitMQ, sistemas de pagamento, IoT	Desacoplamento, alta escalabilidade, reatividade	Dificuldade de rastreamento e consistência
Peer-to-Peer (P2P)	Todos os nós podem ser clientes e servidores	BitTorrent, Blockchain, redes de compartilhamento	Descentralização, tolerância a falhas	Dificuldade de coordenação, segurança e confiabilidade
Baseada em DHT (Distributed Hash Table)	Estrutura de hash distribuída para busca/armazenamento	Kademlia, Chord, DynamoDB	Busca eficiente, escalabilidade massiva	Complexidade de implementação e manutenção

Conclusão

Fundamentos e Requisitos

Escalabilidade, disponibilidade, tolerância a falhas e segurança orientam qualquer arquitetura distribuída.

1. Cliente-Servidor

Modelo simples e centralizado; facilita a segurança, mas cria gargalo e ponto único de falha.

2. Arquitetura em Camadas (N-Tier)

Organização hierárquica que favorece modularidade e manutenção, com custo de latência entre camadas.

3. SOA

Serviços independentes integrados por protocolos padronizados, com boa reutilização e maior complexidade de governança.

4. Microsserviços

Evolução do SOA com serviços pequenos, autônomos e de deploy independente; exige orquestração e monitoramento cuidadosos.

5. Arquitetura Baseada em Eventos

Componentes se comunicam de forma assíncrona por eventos, ganhando desacoplamento e escalabilidade, ao custo de rastreabilidade.

6. Peer-to-Peer (P2P)

Sem servidor central: todo nó é cliente e servidor, o que traz descentralização e tolerância a falhas, com maior desafio de coordenação.

7. Baseada em DHT

Organiza os nós em uma tabela de hash distribuída (ex.: Chord), permitindo busca eficiente e escalabilidade massiva.

Dúvidas?

Obrigado pela atenção!

Prof. Paulo Bressan

